

5th Lab Session

Department of Statistics and Operations Research
School of Mathematics and Statistics
Universitat Politècnica de Catalunya



Outline

- Benders algorithm
- Example of Benders implementation using AMPL
- Lab assignment

Benders algorithm

0. Initializations

1. Solve $(BPr) \rightarrow z_r^*, x_r^*$

2. Solve (Qd) fixing $x = x_r^* \rightarrow (u^*, z^*)$

- a. if (Qd) has a solution compare z_r^* with $z^* = cx_r^* + u^*(h - Tx_r^*)$
 if $z_r^* = z^*$ then
 STOP: optimal solution found
 else

add optimality cut $z \geq cx + u^*(h - Tx)$ to (BPr)

- b. if (Qd) is unbounded then

add feasibility cut $v^*(h - Tx) \leq 0$ to (BPr)
 (and possibly add optimality cut)

3. go to step 1.

.mod: Original problem

```
set ORIG;    # shipment origins (warehouses)
set DEST;    # shipment destinations (stores)

param supply {ORIG} > 0;
param demand {DEST} > 0;

var Build {ORIG} binary;    # 1 iff it is built
param fix_cost {ORIG} > 0;

var Ship {ORIG,DEST} >= 0;  # amounts shipped
param var_cost {ORIG,DEST} > 0;

minimize Total_Cost:
    sum {i in ORIG} fix_cost[i] * Build[i] +
    sum {i in ORIG, j in DEST} var_cost[i,j] * Ship[i,j];

subj to Supply {i in ORIG}:
    sum {j in DEST} Ship[i,j] <= supply[i] * Build[i];

subj to Demand {j in DEST}:
    sum {i in ORIG} Ship[i,j] = demand[j];
```

.mod: Subproblem

```
### SUBPROBLEM ###
```

```
set ORIG;    # shipment origins (warehouses)
set DEST;    # shipment destinations (stores)

param supply {ORIG} > 0;
param demand {DEST} > 0;

param fix_cost {ORIG} > 0;
param var_cost {ORIG,DEST} > 0;

param build {ORIG} binary; # = 1 iff warehouse built at i

param BIG := 1.0e+9;        # objective > BIG ==> unbounded

var Supply_Price {ORIG} <= 0;
var Demand_Price {DEST};

maximize Dual_Ship_Cost:
    sum {i in ORIG} Supply_Price[i] * supply[i] * build[i] +
    sum {j in DEST} Demand_Price[j] * demand[j];

subj to Dual_Ship {i in ORIG, j in DEST}:
    Supply_Price[i] + Demand_Price[j] <= var_cost[i,j];
```

.mod: Subproblem

```
### SUBPROBLEM ###
```

```
set ORIG;    # shipment origins (warehouses)
set DEST;    # shipment destinations (stores)

param supply {ORIG} > 0;
param demand {DEST} > 0;

param fix_cost {ORIG} > 0;
param var_cost {ORIG,DEST} > 0;

param build {ORIG} binary;  # = 1 iff warehouse built at i

param BIG := 1.0e+9;        # objective > BIG ==> unbounded

var Supply_Price {ORIG} <= 0;
var Demand_Price {DEST};

maximize Dual_Ship_Cost:
    sum {i in ORIG} Supply_Price[i] * supply[i] * build[i] +
    sum {j in DEST} Demand_Price[j] * demand[j];

subj to Dual_Ship {i in ORIG, j in DEST}:
    Supply_Price[i] + Demand_Price[j] <= var_cost[i,j];
```

.mod: Subproblem

```
### SUBPROBLEM ###
```

```
set ORIG;    # shipment origins (warehouses)
set DEST;    # shipment destinations (stores)
```

```
param supply {ORIG} > 0;
param demand {DEST} > 0;
```

```
param fix_cost {ORIG} > 0;
param var_cost {ORIG,DEST} > 0;
```

```
param build {ORIG} binary;  # = 1 iff warehouse built at i
```

```
param BIG := 1.0e+9;        # objective > BIG ==> unbounded
```

```
var Supply_Price {ORIG} <= 0;
var Demand_Price {DEST};
```

```
maximize Dual_Ship_Cost:
    sum {i in ORIG} Supply_Price[i] * supply[i] * build[i] +
    sum {j in DEST} Demand_Price[j] * demand[j];
```

```
subj to Dual_Ship {i in ORIG, j in DEST}:
    Supply_Price[i] + Demand_Price[j] <= var_cost[i,j];
```

.mod: Subproblem

```
### SUBPROBLEM ###
```

```
set ORIG;    # shipment origins (warehouses)
set DEST;    # shipment destinations (stores)
```

```
param supply {ORIG} > 0;
param demand {DEST} > 0;
```

```
param fix_cost {ORIG} > 0;
param var_cost {ORIG,DEST} > 0;
```

```
param build {ORIG} binary; # = 1 iff warehouse built at i
```

```
param BIG := 1.0e+9;      # objective > BIG ==> unbounded
```

```
var Supply_Price {ORIG} <= 0;
var Demand_Price {DEST};
```

```
maximize Dual_Ship_Cost:
```

```
    sum {i in ORIG} Supply_Price[i] * supply[i] * build[i] +
    sum {j in DEST} Demand_Price[j] * demand[j];
```

```
subj to Dual_Ship {i in ORIG, j in DEST}:
```

```
    Supply_Price[i] + Demand_Price[j] <= var_cost[i,j];
```


.mod: Master Problem

```
### MASTER PROBLEM ###
```

```
param nCUT >= 0 integer;
param cut_type {1..nCUT} symbolic within {"point","ray"};
param supply_price {ORIG,1..nCUT} <= 0.000001;
param demand_price {DEST,1..nCUT};

var Build {ORIG} binary;    # = 1 iff warehouse built at i
var Max_Ship_Cost;

minimize Total_Cost:
    sum {i in ORIG} fix_cost[i] * Build[i] + Max_Ship_Cost;

subj to Cut_Dfn {k in 1..nCUT}:
    if cut_type[k] = "point" then Max_Ship_Cost >=
        sum {i in ORIG} supply_price[i,k] * supply[i] * Build[i] +
        sum {j in DEST} demand_price[j,k] * demand[j];
```

.mod: Master Problem

```
### MASTER PROBLEM ###
```

```
param nCUT >= 0 integer;  
param cut_type {1..nCUT} symbolic within {"point","ray"};  
param supply_price {ORIG,1..nCUT} <= 0.000001;  
param demand_price {DEST,1..nCUT};
```

```
var Build {ORIG} binary;    # = 1 iff warehouse built at i  
var Max_Ship_Cost;
```

```
minimize Total_Cost:  
    sum {i in ORIG} fix_cost[i] * Build[i] + Max_Ship_Cost;
```

```
subj to Cut_Defn {k in 1..nCUT}:  
    if cut_type[k] = "point" then Max_Ship_Cost >=  
        sum {i in ORIG} supply_price[i,k] * supply[i] * Build[i] +  
        sum {j in DEST} demand_price[j,k] * demand[j];
```

.mod: Master Problem

```
### MASTER PROBLEM ###
```

```
param nCUT >= 0 integer;  
param cut_type {1..nCUT} symbolic within {"point","ray"};  
param supply_price {ORIG,1..nCUT} <= 0.000001;  
param demand_price {DEST,1..nCUT};
```

```
var Build {ORIG} binary;    # = 1 iff warehouse built at i  
var Max_Ship_Cost;
```

```
minimize Total_Cost:  
    sum {i in ORIG} fix_cost[i] * Build[i] + Max_Ship_Cost;  
  
subj to Cut_Defn {k in 1..nCUT}:  
    if cut_type[k] = "point" then Max_Ship_Cost >=  
        sum {i in ORIG} supply_price[i,k] * supply[i] * Build[i] +  
        sum {j in DEST} demand_price[j,k] * demand[j];
```

.mod: Master Problem

```
### MASTER PROBLEM ###
```

```
param nCUT >= 0 integer;  
param cut_type {1..nCUT} symbolic within {"point","ray"};  
param supply_price {ORIG,1..nCUT} <= 0.000001;  
param demand_price {DEST,1..nCUT};
```

```
var Build {ORIG} binary;    # = 1 iff warehouse built at i  
var Max_Ship_Cost;
```

```
minimize Total_Cost:  
    sum {i in ORIG} fix_cost[i] * Build[i] + Max_Ship_Cost;  
  
subj to Cut_Dfn {k in 1..nCUT}:  
    if cut_type[k] = "point" then Max_Ship_Cost >=  
        sum {i in ORIG} supply_price[i,k] * supply[i] * Build[i] +  
        sum {j in DEST} demand_price[j,k] * demand[j];
```

.run: Initialization

```
model trnloc1d.mod;  
data trnloc1.dat;  
  
option solver cplex;  
option cplex_options 'mipdisplay 2 mipinterval 100 primal';  
  
option omit_zero_rows 1;  
option display_eps .000001;  
  
problem Master: Build, Max_Ship_Cost, Total_Cost, Cut_Dfn;  
problem Sub: Supply_Price, Demand_Price, Dual_Ship_Cost, Dual_Ship;  
  
suffix unbdd OUT;  
  
let nCUT := 0;  
let Max_Ship_Cost := 0;  
let {i in ORIG} build[i] := 1;  
  
param GAP default Infinity;
```

.run: Initialization

```
model trnloc1d.mod;
data trnloc1.dat;

option solver cplex;
option cplex_options 'mipdisplay 2 mipinterval 100 primal';

option omit_zero_rows 1;
option display_eps .000001;

problem Master: Build, Max_Ship_Cost, Total_Cost, Cut_Defn;
problem Sub: Supply_Price, Demand_Price, Dual_Ship_Cost, Dual_Ship;

suffix unbdd OUT;

let nCUT := 0;
let Max_Ship_Cost := 0;
let {i in ORIG} build[i] := 1;

param GAP default Infinity;
```

.run: Initialization

```
model trnloc1d.mod;
data trnloc1.dat;

option solver cplex;
option cplex_options 'mipdisplay 2 mipinterval 100 primal';

option omit_zero_rows 1;
option display_eps .000001;

problem Master: Build, Max_Ship_Cost, Total_Cost, Cut_Dfn;
problem Sub: Supply_Price, Demand_Price, Dual_Ship_Cost, Dual_Ship;

suffix unbdd OUT;

let nCUT := 0;
let Max_Ship_Cost := 0;
let {i in ORIG} build[i] := 1;

param GAP default Infinity;
```

.run: Initialization

```
model trnloc1d.mod;
data trnloc1.dat;

option solver cplex;
option cplex_options 'mipdisplay 2 mipinterval 100 primal';

option omit_zero_rows 1;
option display_eps .000001;

problem Master: Build, Max_Ship_Cost, Total_Cost, Cut_Defn;
problem Sub: Supply_Price, Demand_Price, Dual_Ship_Cost, Dual_Ship;

suffix unbdd OUT;

let nCUT := 0;
let Max_Ship_Cost := 0;
let {i in ORIG} build[i] := 1;

param GAP default Infinity;
```


.run: Iterations

```
repeat { printf "\nITERATION %d\n\n", nCUT+1;

    solve Sub;
    printf "\n";

    if Sub.result = "unbounded" then { printf "UNBOUNDED\n";
        let nCUT := nCUT + 1;
        let cut_type[nCUT] := "ray";
        let {i in ORIG} supply_price[i,nCUT] := Supply_Price[i].unbdd;
        let {j in DEST} demand_price[j,nCUT] := Demand_Price[j].unbdd;
    }

    else {
        if Dual_Ship_Cost <= Max_Ship_Cost + 0.00001 then break;

        let GAP := min (GAP, Dual_Ship_Cost - Max_Ship_Cost);
        option display_1col 0;
        display GAP, Dual_Ship;
        let nCUT := nCUT + 1;
        let cut_type[nCUT] := "point";
        let {i in ORIG} supply_price[i,nCUT] := Supply_Price[i];
        let {j in DEST} demand_price[j,nCUT] := Demand_Price[j];
    }

    printf "\nRE-SOLVING MASTER PROBLEM\n\n";

    solve Master;
    printf "\n";
    option display_1col 20;
    display Build;

    let {i in ORIG} build[i] := Build[i];
};
```

.run: Iterations

```
repeat { printf "\nITERATION %d\n\n", nCUT+1;

  solve Sub;
  printf "\n";

  if Sub.result = "unbounded" then { printf "UNBOUNDED\n";
    let nCUT := nCUT + 1;
    let cut_type[nCUT] := "ray";
    let {i in ORIG} supply_price[i,nCUT] := Supply_Price[i].unbdd;
    let {j in DEST} demand_price[j,nCUT] := Demand_Price[j].unbdd;
  }

  else {
    if Dual_Ship_Cost <= Max_Ship_Cost + 0.00001 then break;

    let GAP := min (GAP, Dual_Ship_Cost - Max_Ship_Cost);
    option display_1col 0;
    display GAP, Dual_Ship;
    let nCUT := nCUT + 1;
    let cut_type[nCUT] := "point";
    let {i in ORIG} supply_price[i,nCUT] := Supply_Price[i];
    let {j in DEST} demand_price[j,nCUT] := Demand_Price[j];
  }

  printf "\nRE-SOLVING MASTER PROBLEM\n\n";

  solve Master;
  printf "\n";
  option display_1col 20;
  display Build;

  let {i in ORIG} build[i] := Build[i];
};
```

.run: Iterations

```
repeat { printf "\nITERATION %d\n\n", nCUT+1;

    solve Sub;
    printf "\n";

    if Sub.result = "unbounded" then { printf "UNBOUNDED\n";
        let nCUT := nCUT + 1;
        let cut_type[nCUT] := "ray";
        let {i in ORIG} supply_price[i,nCUT] := Supply_Price[i].unbdd;
        let {j in DEST} demand_price[j,nCUT] := Demand_Price[j].unbdd;
    }

    else {
        if Dual_Ship_Cost <= Max_Ship_Cost + 0.00001 then break;

        let GAP := min (GAP, Dual_Ship_Cost - Max_Ship_Cost);
        option display_1col 0;
        display GAP, Dual_Ship;
        let nCUT := nCUT + 1;
        let cut_type[nCUT] := "point";
        let {i in ORIG} supply_price[i,nCUT] := Supply_Price[i];
        let {j in DEST} demand_price[j,nCUT] := Demand_Price[j];
    }

    printf "\nRE-SOLVING MASTER PROBLEM\n\n";

    solve Master;
    printf "\n";
    option display_1col 20;
    display Build;

    let {i in ORIG} build[i] := Build[i];
};
```

.run: Iterations

```
repeat { printf "\nITERATION %d\n\n", nCUT+1;

  solve Sub;
  printf "\n";

  if Sub.result = "unbounded" then { printf "UNBOUNDED\n";
    let nCUT := nCUT + 1;
    let cut_type[nCUT] := "ray";
    let {i in ORIG} supply_price[i,nCUT] := Supply_Price[i].unbdd;
    let {j in DEST} demand_price[j,nCUT] := Demand_Price[j].unbdd;
  }

  else {
    if Dual_Ship_Cost <= Max_Ship_Cost + 0.00001 then break;

    let GAP := min (GAP, Dual_Ship_Cost - Max_Ship_Cost);
    option display_1col 0;
    display GAP, Dual_Ship;
    let nCUT := nCUT + 1;
    let cut_type[nCUT] := "point";
    let {i in ORIG} supply_price[i,nCUT] := Supply_Price[i];
    let {j in DEST} demand_price[j,nCUT] := Demand_Price[j];
  }

  printf "\nRE-SOLVING MASTER PROBLEM\n\n";

  solve Master;
  printf "\n";
  option display_1col 20;
  display Build;

  let {i in ORIG} build[i] := Build[i];
};
```

.run: Iterations

```
repeat { printf "\nITERATION %d\n\n", nCUT+1;

  solve Sub;
  printf "\n";

  if Sub.result = "unbounded" then { printf "UNBOUNDED\n";
    let nCUT := nCUT + 1;
    let cut_type[nCUT] := "ray";
    let {i in ORIG} supply_price[i,nCUT] := Supply_Price[i].unbdd;
    let {j in DEST} demand_price[j,nCUT] := Demand_Price[j].unbdd;
  }

  else {
    if Dual_Ship_Cost <= Max_Ship_Cost + 0.00001 then break;

    let GAP := min (GAP, Dual_Ship_Cost - Max_Ship_Cost);
    option display_1col 0;
    display GAP, Dual_Ship;
    let nCUT := nCUT + 1;
    let cut_type[nCUT] := "point";
    let {i in ORIG} supply_price[i,nCUT] := Supply_Price[i];
    let {j in DEST} demand_price[j,nCUT] := Demand_Price[j];
  }

  printf "\nRE-SOLVING MASTER PROBLEM\n\n";

  solve Master;
  printf "\n";
  option display_1col 20;
  display Build;

  let {i in ORIG} build[i] := Build[i];
};
```

.run: Iterations

```
repeat { printf "\nITERATION %d\n\n", nCUT+1;

  solve Sub;
  printf "\n";

  if Sub.result = "unbounded" then { printf "UNBOUNDED\n";
    let nCUT := nCUT + 1;
    let cut_type[nCUT] := "ray";
    let {i in ORIG} supply_price[i,nCUT] := Supply_Price[i].unbdd;
    let {j in DEST} demand_price[j,nCUT] := Demand_Price[j].unbdd;
  }

  else {
    if Dual_Ship_Cost <= Max_Ship_Cost + 0.00001 then break;

    let GAP := min (GAP, Dual_Ship_Cost - Max_Ship_Cost);
    option display_1col 0;
    display GAP, Dual_Ship;
    let nCUT := nCUT + 1;
    let cut_type[nCUT] := "point";
    let {i in ORIG} supply_price[i,nCUT] := Supply_Price[i];
    let {j in DEST} demand_price[j,nCUT] := Demand_Price[j];
  }

  printf "\nRE-SOLVING MASTER PROBLEM\n\n";

  solve Master;
  printf "\n";
  option display_1col 20;
  display Build;

  let {i in ORIG} build[i] := Build[i];
};
```

.run: Iterations

```
repeat { printf "\nITERATION %d\n\n", nCUT+1;

  solve Sub;
  printf "\n";

  if Sub.result = "unbounded" then { printf "UNBOUNDED\n";
    let nCUT := nCUT + 1;
    let cut_type[nCUT] := "ray";
    let {i in ORIG} supply_price[i,nCUT] := Supply_Price[i].unbdd;
    let {j in DEST} demand_price[j,nCUT] := Demand_Price[j].unbdd;
  }

  else {
    if Dual_Ship_Cost <= Max_Ship_Cost + 0.00001 then break;

    let GAP := min (GAP, Dual_Ship_Cost - Max_Ship_Cost);
    option display_1col 0;
    display GAP, Dual_Ship;
    let nCUT := nCUT + 1;
    let cut_type[nCUT] := "point";
    let {i in ORIG} supply_price[i,nCUT] := Supply_Price[i];
    let {j in DEST} demand_price[j,nCUT] := Demand_Price[j];
  }

  printf "\nRE-SOLVING MASTER PROBLEM\n\n";

  solve Master;
  printf "\n";
  option display_1col 20;
  display Build;

  let {i in ORIG} build[i] := Build[i];
};
```